

Yocto Embedded Linux Build — Detailed Purpose of Every Command and Internal Flow

Prepared from your uploaded Yocto training notes.

WHAT IS YOCTO?

Yocto Project is an industrial embedded Linux build framework used to create:

- Embedded Linux firmware
 - Custom Linux distributions
 - BMC/iLO firmware
 - Automotive Linux systems
 - Router operating systems
 - IoT firmware platforms
-

REAL INDUSTRY USAGE

Yocto is used in:

Company/Industry	Usage
Hewlett Packard Enterprise	iLO/BMC firmware
Cisco	Router operating systems
Automotive ECUs	Vehicle Linux platforms
Intel	Embedded Linux BSP

Samsung

Embedded devices

NVIDIA

Jetson Linux

Industrial IoT

Firmware systems

MAIN GOAL OF YOCTO

Source Code



BitBake Recipes



Cross Compilation



Linux Kernel Build



Root Filesystem Generation



Embedded Firmware Image



QEMU / Hardware Boot

COMPLETE YOCTO ENGINEERING FLOW

Ubuntu Host Machine



Install Yocto Dependencies



Download Poky Source



Initialize Build Environment



Configure Build System



BitBake Build Engine



Cross Compile Packages



↓
Build Linux Kernel
↓
Generate RootFS
↓
Generate Firmware Image
↓
Boot in QEMU

STEP 1 — INSTALL YOCTO DEPENDENCIES

```
sudo apt update && sudo apt install -y \  
gawk wget git diffstat unzip texinfo gcc \  
build-essential chrpath socat cpio \  
python3 python3-pip python3-pexpect \  
xz-utils debianutils iputils-ping \  
python3-git python3-jinja2 \  
libSDL2-dev xterm \  
libegl1-mesa-dev pylint
```

MAIN PURPOSE

Installs all required Yocto build tools.

WHAT HAPPENS INTERNALLY

APT Package Manager
↓
Downloads package metadata
↓
Downloads dependencies



↓
Installs tools/libraries
↓
Prepares Ubuntu for Yocto builds

PURPOSE OF IMPORTANT PACKAGES

Package	Purpose
<code>gawk</code>	Text processing
<code>wget</code>	Download files
<code>git</code>	Source code management
<code>diffstat</code>	Patch statistics
<code>unzip</code>	Extract ZIP archives
<code>gcc</code>	Native compiler
<code>build-essential</code>	GCC + Make tools
<code>chrpath</code>	Modify binary RPATH
<code>socat</code>	Socket communication
<code>cpio</code>	RootFS archive creation
<code>python3</code>	Yocto scripts runtime
<code>python3-pip</code>	Python package installer
<code>python3-pexpect</code>	Automation scripts
<code>xz-utils</code>	Extract tar.xz
<code>iputils-ping</code>	Network testing



<code>python3-jinja2</code>	Template engine
<code>libSDL2-dev</code>	QEMU graphics
<code>xterm</code>	Terminal emulator
<code>libegl1-mesa-d</code> <code>ev</code>	OpenGL support
<code>pylint</code>	Python validation

WHY THIS STEP IMPORTANT

Without these tools:

BitBake build system
will fail

because Yocto depends heavily on:

- Python
 - compilers
 - archive tools
 - patch tools
 - filesystem generators
-

STEP 2 — CLONE POKY SOURCE

`git clone git://git.yoctoproject.org/poky`

WHAT IS POKY?

Poky is the official Yocto reference distribution.



Contains:

Component	Purpose
BitBake	Build engine
Meta layers	Recipes/configurations
Toolchains	Cross compilation
QEMU support	Virtual testing

WHAT HAPPENS INTERNALLY

Git Server
↓
Downloads commits
↓
Downloads layers
↓
Creates local source tree

OUTPUT

poky/

IMPORTANT POKY FOLDERS

Folder	Purpose
<code>meta/</code>	Core recipes
<code>meta-poky/</code>	Poky distribution

`meta-yocto-bs` BSP support
`p/`

`bitbake/` Build engine

`scripts/` Build utilities

STEP 3 — MOVE INTO POKY

`cd poky`

PURPOSE

Moves terminal into Yocto source tree.

WHY IMPORTANT

All Yocto commands execute from this workspace.

STEP 4 — CHECKOUT STABLE BRANCH

`git checkout kirkstone`

PURPOSE

Switches to stable Yocto release branch.

WHY IMPORTANT

kirkstone is:

- Long-Term Support (LTS)
 - Stable production branch
 - Common in embedded products
-

WHAT HAPPENS INTERNALLY

Git repository



Switches branch



Loads kirkstone source state

STEP 5 — INITIALIZE YOCTO ENVIRONMENT

source oe-init-build-env

VERY IMPORTANT STEP

Initializes Yocto build environment.

WHAT HAPPENS INTERNALLY

Exports environment variables



Creates build directory



Configures PATH



Initializes BitBake

IMPORTANT OUTPUT

build/

Generated build workspace.

IMPORTANT GENERATED FILES

File	Purpose
<code>conf/local.conf</code>	Main build configuration
<code>conf/bblayers.conf</code>	Layer configuration
<code>nf</code>	

WHY IMPORTANT

Without this:

bitbake command will fail

STEP 6 — BUILD EMBEDDED LINUX IMAGE

bitbake core-image-minimal

THIS IS THE MAIN YOCTO BUILD STEP

MAIN PURPOSE

Builds complete embedded Linux firmware image.

WHAT IS **core-image-minimal**?

Minimal embedded Linux image containing:

- Linux kernel
 - BusyBox
 - Root filesystem
 - Init system
 - Basic shell
-

WHAT HAPPENS INTERNALLY

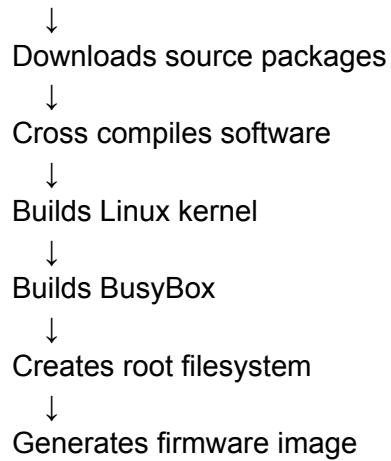
BitBake



Reads recipes (.bb)



Resolves dependencies



IMPORTANT YOCTO COMPONENTS

Component	Purpose
BitBake	Build engine
Recipes (.bb)	Package instructions
Layers	Package collections
Toolchain	Cross compiler
RootFS	Embedded filesystem

GENERATED OUTPUT FILES

Located in:

tmp/deploy/images/qemux86-64/

IMPORTANT GENERATED FILES

File	Purpose
<code>bzImage</code>	Linux kernel
<code>.ext4</code>	Root filesystem
<code>.tar.bz2</code>	Compressed rootFS
<code>.manifest</code>	Installed packages
<code>.qemuboot.conf</code>	QEMU boot config

COMPLETE INTERNAL BUILD FLOW

Recipes



Cross Compiler



Package Builds



Kernel Build



RootFS Assembly



Filesystem Images



Embedded Firmware

STEP 7 — BOOT YOCTO IMAGE IN QEMU

```
runqemu qemux86-64
```

PURPOSE

Boots generated embedded Linux firmware.

WHAT HAPPENS INTERNALLY

QEMU Emulator
↓
Loads Linux kernel
↓
Loads root filesystem
↓
Starts virtual hardware
↓
Linux boot process begins

COMPLETE BOOT FLOW

QEMU BIOS
↓
Linux Kernel
↓
Mount RootFS
↓
Run init/systemd
↓
BusyBox shell
↓
Embedded Linux running

STEP 8 — VERIFY EMBEDDED LINUX

Check Kernel

uname -a

PURPOSE

Displays:

- Linux kernel version
 - architecture
 - hostname
-

STEP 9 — CHECK PID 1

ps -p 1

PURPOSE

Checks init process.

WHY IMPORTANT

PID 1 controls:

- service management
- process startup
- system initialization

Usually:

systemd
or
init

STEP 10 — CHECK BUSYBOX

busybox

PURPOSE

Displays BusyBox applets.

WHAT IS BUSYBOX?

Single executable providing:

- ls
 - cp
 - sh
 - ps
 - grep
 - mount
-

INTERNAL BUSYBOX FLOW

busybox binary
↓
detects command
↓
runs internal applet

STEP 11 — CHECK FILESYSTEMS

mount

PURPOSE

Shows mounted filesystems.

WHY IMPORTANT

Confirms:

- RootFS mounted
 - proc/sys mounted
 - virtual filesystems working
-

STEP 12 — CHECK NETWORK

ip addr

PURPOSE

Displays network interfaces.

INTERNALLY

Kernel networking stack



Interface configuration



IP address display

STEP 13 — CREATE CUSTOM YOCTO LAYER

```
bitbake-layers create-layer ../meta-ilo
```

PURPOSE

Creates custom Yocto layer.

WHY IMPORTANT

Used to add:

- custom packages
 - services
 - drivers
 - firmware applications
-

GENERATED STRUCTURE

meta-ilo/

STEP 14 — ADD CUSTOM LAYER

```
bitbake-layers add-layer ../meta-ilo
```

PURPOSE

Registers custom layer into Yocto build system.

WHAT HAPPENS INTERNALLY

Updates bblayers.conf



BitBake can now read custom recipes

STEP 15 — CREATE CUSTOM SERVICE RECIPE

```
nano ../meta-ilo/recipes-ilo/ilo-service/ilo-service.bb
```

PURPOSE

Creates BitBake recipe.

WHAT IS A RECIPE?

Recipe describes:

- source files
 - build process
 - installation path
 - package metadata
-

RECIPE INTERNAL FLOW

Recipe (.bb)



BitBake parser



Build instructions



Package generation

STEP 16 — REBUILD IMAGE

bitbake core-image-minimal

PURPOSE

Rebuilds firmware including custom package.

INTERNALLY

BitBake



Detects custom layer



Builds ilo-service package



Adds package to rootFS



Generates new firmware image

STEP 17 — RUN UPDATED IMAGE

`runqemu qemux86-64`

PURPOSE

Boot updated firmware image.

VERIFY CUSTOM SERVICE

which ilo-service

PURPOSE

Checks if package installed correctly.

EXPECTED OUTPUT

/usr/bin/ilo-service

COMPLETE YOCTO MENTAL MODEL

Layers



Recipes



BitBake



Cross Compilation



Kernel Build



BusyBox Build



Root Filesystem



Firmware Image



QEMU Boot

CORE ENGINEERING CONCEPTS YOU LEARNED

Concept	Meaning
Yocto	Embedded Linux build framework
Poky	Reference Yocto distro
BitBake	Build engine

Recipe	Package build instruction
Layer	Collection of recipes
RootFS	Embedded filesystem
QEMU	Hardware emulator
BusyBox	Embedded userspace
Cross Compilation	Build for another CPU
Firmware Image	Final embedded Linux OS

FINAL INDUSTRY UNDERSTANDING

This exact workflow is used in:

- iLO/BMC firmware engineering
- Router operating systems
- Automotive Linux
- Embedded BSP development
- ARM Linux platforms
- Industrial IoT firmware
- Linux-based embedded products